

Last updated on **Dec 18, 2024**

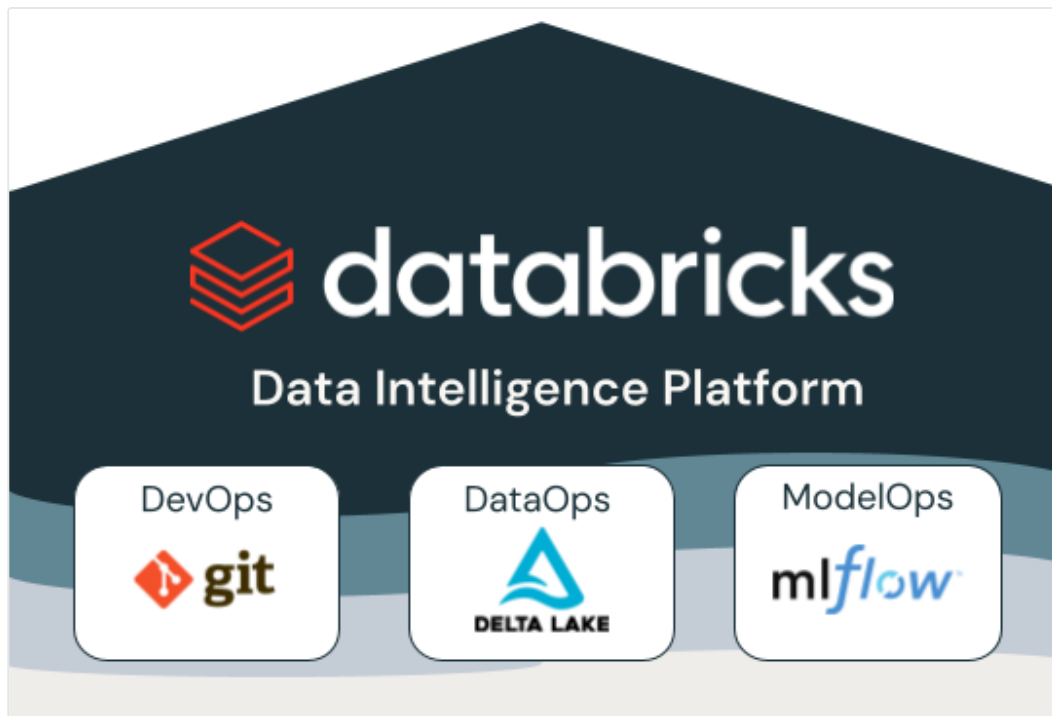
MLOps workflows on Databricks

This article describes how you can use MLOps on the Databricks platform to optimize the performance and long-term efficiency of your machine learning (ML) systems. It includes general recommendations for an MLOps architecture and describes a generalized workflow using the Databricks platform that you can use as a model for your ML development-to-production process. For modifications of this workflow for LLMOps applications, see [LLMOps workflows](#).

For more details, see [The Big Book of MLOps](#).

What is MLOps?

MLOps is a set of processes and automated steps for managing code, data, and models to improve performance, stability, and long-term efficiency of ML systems. It combines DevOps, DataOps, and ModelOps.



ML assets such as code, data, and models are developed in stages that progress from early development stages that do not have tight access limitations and are not rigorously tested, through an intermediate testing stage, to a final production stage that is tightly controlled. The Databricks platform lets you manage these assets on a single platform with unified access control. You can develop data applications and ML applications on the same platform, reducing the risks and delays associated with moving data around.

General recommendations for MLOps

This section includes some general recommendations for MLOps on Databricks with links for more information.

Create a separate environment for each stage

An execution environment is the place where models and data are created or consumed by code. Each execution environment consists of compute instances, their runtimes and libraries, and automated jobs.

Databricks recommends creating separate environments for the different stages of ML code and model development with clearly defined transitions between stages. The

workflow described in this article follows this process, using the common names for the stages:

- [Development](#)
- [Staging](#)
- [Production](#)

Other configurations can also be used to meet the specific needs of your organization.

Access control and versioning

Access control and versioning are key components of any software operations process. Databricks recommends the following:

- **Use Git for version control.** Pipelines and code should be stored in Git for version control. Moving ML logic between stages can then be interpreted as moving code from the development branch, to the staging branch, to the release branch. Use [Databricks Git folders](#) to integrate with your Git provider and sync notebooks and source code with Databricks workspaces. Databricks also provides additional tools for Git integration and version control; see [Local development tools](#).
- **Store data in a lakehouse architecture using Delta tables.** Data should be stored in a [lakehouse architecture](#) in your cloud account. Both raw data and feature tables should be stored as [Delta tables](#) with access controls to determine who can read and modify them.
- **Manage model development with MLflow.** You can use [MLflow](#) to track the model development process and save code snapshots, model parameters, metrics, and other metadata.
- **Use Models in Unity Catalog to manage the model lifecycle.** Use [Models in Unity Catalog](#) to manage model versioning, governance, and deployment status.

Deploy code, not models

In most situations, Databricks recommends that during the ML development process, you promote *code*, rather than *models*, from one environment to the next. Moving project assets this way ensures that all code in the ML development process goes through the same code review and integration testing processes. It also ensures that

the production version of the model is trained on production code. For a more detailed discussion of the options and trade-offs, see [Model deployment patterns](#).

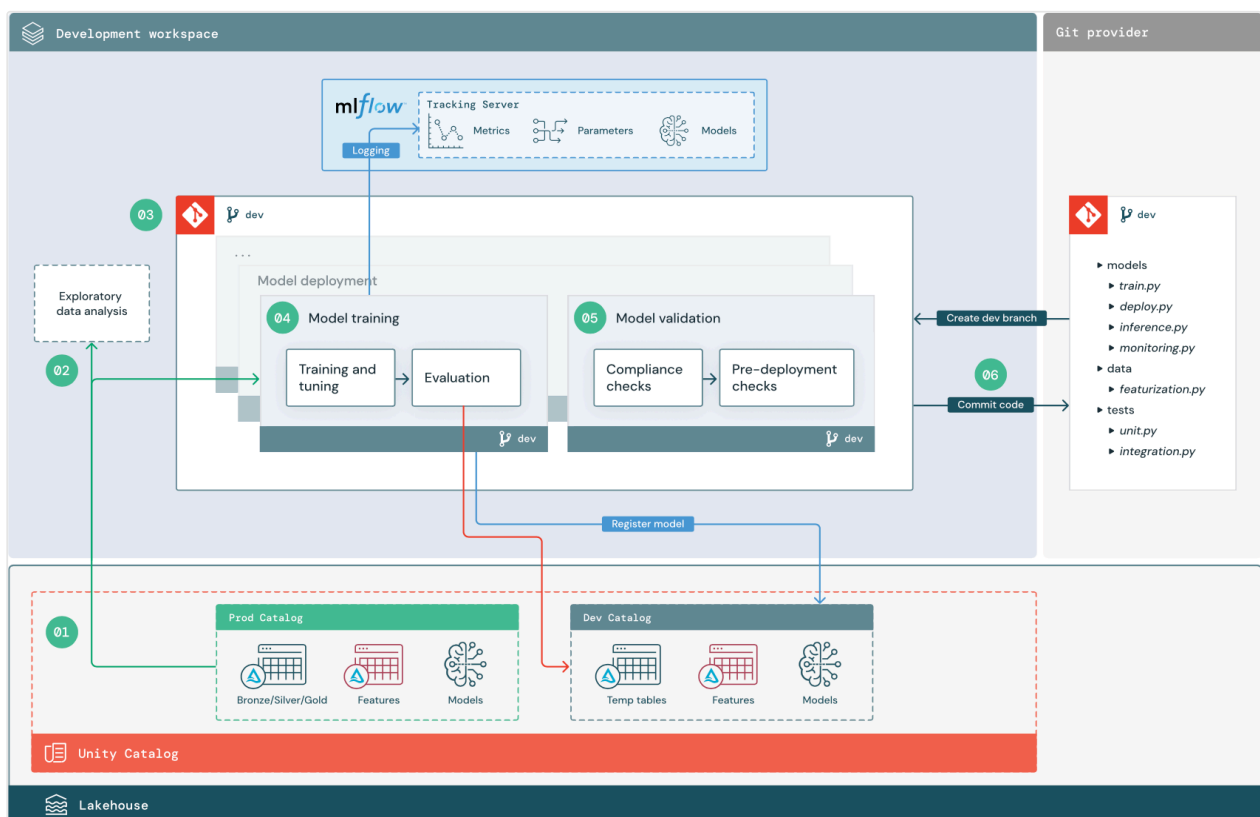
Recommended MLOps workflow

The following sections describe a typical MLOps workflow, covering each of the three stages: development, staging, and production.

This section uses the terms “data scientist” and “ML engineer” as archetypal personas; specific roles and responsibilities in the MLOps workflow will vary between teams and organizations.

Development stage

The focus of the development stage is experimentation. Data scientists develop features and models and run experiments to optimize model performance. The output of the development process is ML pipeline code that can include feature computation, model training, inference, and monitoring.



Ask Genie

The numbered steps correspond to the numbers shown in the diagram.

1. Data sources

The development environment is represented by the dev catalog in Unity Catalog. Data scientists have read-write access to the dev catalog as they create temporary data and feature tables in the development workspace. Models created in the development stage are registered to the dev catalog.

Ideally, data scientists working in the development workspace also have read-only access to production data in the prod catalog. Allowing data scientists read access to production data, inference tables, and metric tables in the prod catalog enables them to analyze current production model predictions and performance. Data scientists should also be able to load production models for experimentation and analysis.

If it is not possible to grant read-only access to the prod catalog, a snapshot of production data can be written to the dev catalog to enable data scientists to develop and evaluate project code.

2. Exploratory data analysis (EDA)

Data scientists explore and analyze data in an interactive, iterative process using notebooks. The goal is to assess whether the available data has the potential to solve the business problem. In this step, the data scientist begins identifying data preparation and featurization steps for model training. This ad hoc process is generally not part of a pipeline that will be deployed in other execution environments.

[AutoML](#) accelerates this process by generating baseline models for a dataset. AutoML performs and records a set of trials and provides a Python notebook with the source code for each trial run, so you can review, reproduce, and modify the code. AutoML also calculates summary statistics on your dataset and saves this information in a notebook that you can review.

3. Code

The code repository contains all of the pipelines, modules, and other project files for an ML project. Data scientists create new or updated pipelines in a development ("dev") branch of the project repository. Starting from EDA and the initial phases of a project, data scientists should work in a repository to share code and track changes.

4. Train model (development)

Data scientists develop the model training pipeline in the development environment using tables from the dev or prod catalogs.

This pipeline includes 2 tasks:

- **Training and tuning.** The training process logs model parameters, metrics, and artifacts to the MLflow Tracking server. After training and tuning hyperparameters, the final model artifact is logged to the tracking server to record a link between the model, the input data it was trained on, and the code used to generate it.
- **Evaluation.** Evaluate model quality by testing on held-out data. The results of these tests are logged to the MLflow Tracking server. The purpose of evaluation is to determine if the newly developed model performs better than the current production model. Given sufficient permissions, any production model registered to the prod catalog can be loaded into the development workspace and compared against a newly trained model.

If your organization's governance requirements include additional information about the model, you can save it using [MLflow tracking](#). Typical artifacts are plain text descriptions and model interpretations such as plots produced by SHAP. Specific governance requirements may come from a data governance officer or business stakeholders.

The output of the model training pipeline is an ML model artifact stored in the MLflow Tracking server for the development environment. If the pipeline is executed in the staging or production workspace, the model artifact is stored in the MLflow Tracking server for that workspace.

When the model training is complete, register the model to Unity Catalog. Set up your pipeline code to register the model to the catalog corresponding to the environment that the model pipeline was executed in; in this example, the dev catalog.

With the recommended architecture, you deploy a multitask Databricks workflow in which the first task is the model training pipeline, followed by model validation and model deployment tasks. The model training task yields a model URI that the model validation task can use. You can use [task values](#) to pass this URI to the model.

5. Validate and deploy model (development)

In addition to the model training pipeline, other pipelines such as model validation and model deployment pipelines are developed in the development environment.

- **Model validation.** The model validation pipeline takes the model URI from the model training pipeline, loads the model from Unity Catalog, and runs validation checks.

Validation checks depend on the context. They can include fundamental checks such as confirming format and required metadata, and more complex checks that might be required for highly regulated industries, such as predefined compliance checks and confirming model performance on selected data slices.

The primary function of the model validation pipeline is to determine whether a model should proceed to the deployment step. If the model passes pre-deployment checks, it can be assigned the “Challenger” alias in Unity Catalog. If the checks fail, the process ends. You can configure your workflow to notify users of a validation failure. See [Add notifications on a job](#).

- **Model deployment.** The model deployment pipeline typically either directly promotes the newly trained “Challenger” model to “Champion” status using an alias update, or facilitates a comparison between the existing “Champion” model and the new “Challenger” model. This pipeline can also set up any required inference infrastructure, such as Model Serving endpoints. For a detailed discussion of the steps involved in the model deployment pipeline, see [Production](#).

6. Commit code

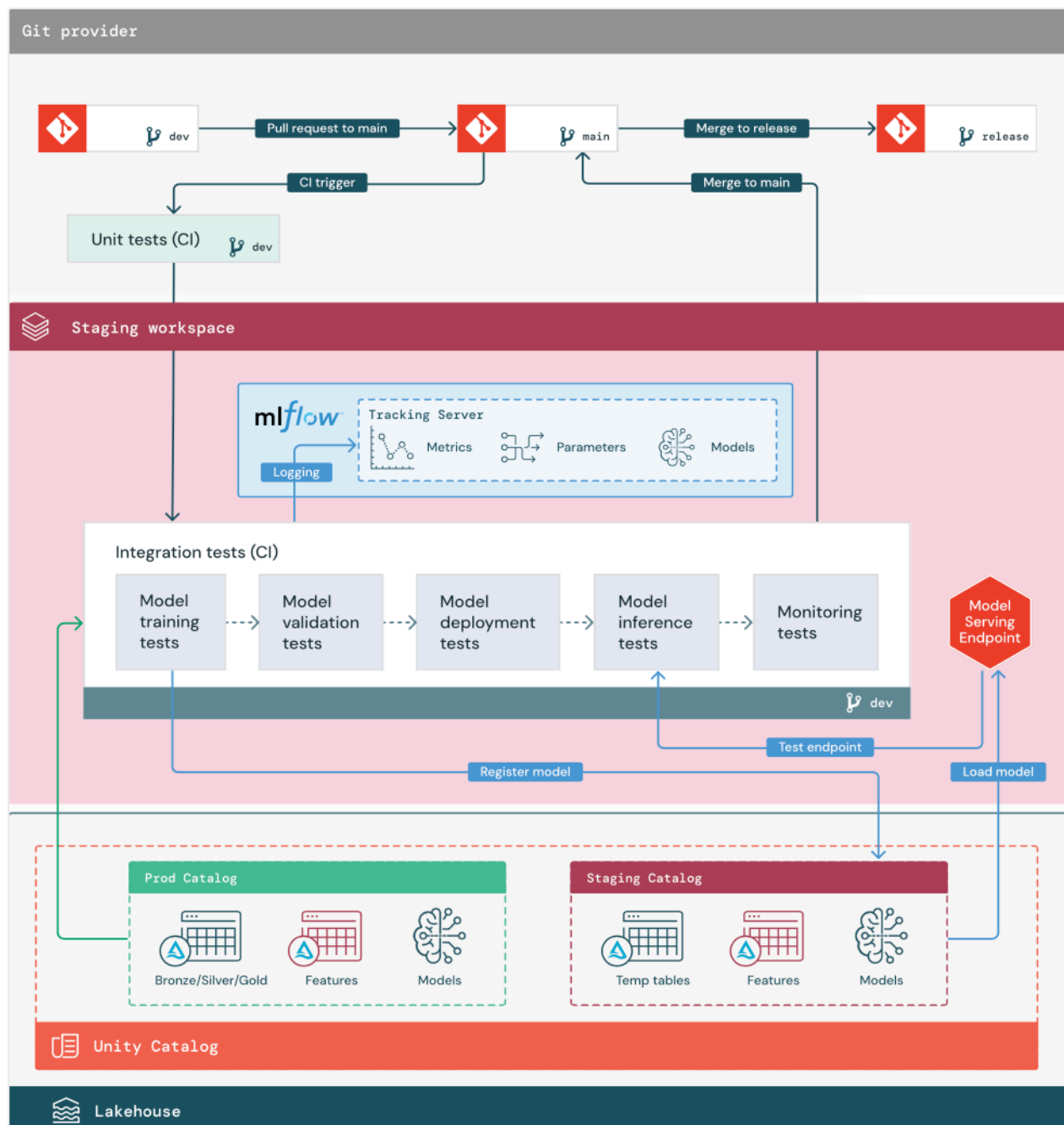
After developing code for training, validation, deployment and other pipelines, the data scientist or ML engineer commits the dev branch changes into source control.

Staging stage

The focus of this stage is testing the ML pipeline code to ensure it is ready for production. All of the ML pipeline code is tested in this stage, including code for model

training as well as feature engineering pipelines, inference code, and so on.

ML engineers create a CI pipeline to implement the unit and integration tests run in this stage. The output of the staging process is a release branch that triggers the CI/CD system to start the production stage.



1. Data

The staging environment should have its own catalog in Unity Catalog for testing ML pipelines and registering models to Unity Catalog. This catalog is shown as the “staging” catalog in the diagram. Assets written to this catalog are generally temporary

and only retained until testing is complete. The development environment may also require access to the staging catalog for debugging purposes.

2. Merge code

Data scientists develop the model training pipeline in the development environment using tables from the development or production catalogs.

- **Pull request.** The deployment process begins when a pull request is created against the main branch of the project in source control.
- **Unit tests (CI).** The pull request automatically builds source code and triggers unit tests. If unit tests fail, the pull request is rejected.

Unit tests are part of the software development process and are continuously executed and added to the codebase during the development of any code. Running unit tests as part of a CI pipeline ensures that changes made in a development branch do not break existing functionality.

3. Integration tests (CI)

The CI process then runs the integration tests. Integration tests run all pipelines (including feature engineering, model training, inference, and monitoring) to ensure that they function correctly together. The staging environment should match the production environment as closely as is reasonable.

If you are deploying an ML application with real-time inference, you should create and test serving infrastructure in the staging environment. This involves triggering the model deployment pipeline, which creates a serving endpoint in the staging environment and loads a model.

To reduce the time required to run integration tests, some steps can trade off between fidelity of testing and speed or cost. For example, if models are expensive or time-consuming to train, you might use small subsets of data or run fewer training iterations. For model serving, depending on production requirements, you might do full-scale load testing in integration tests, or you might just test small batch jobs or requests to a temporary endpoint.

4. Merge to staging branch

If all tests pass, the new code is merged into the main branch of the project. If tests fail, the CI/CD system should notify users and post results on the pull request.

You can schedule periodic integration tests on the main branch. This is a good idea if the branch is updated frequently with concurrent pull requests from multiple users.

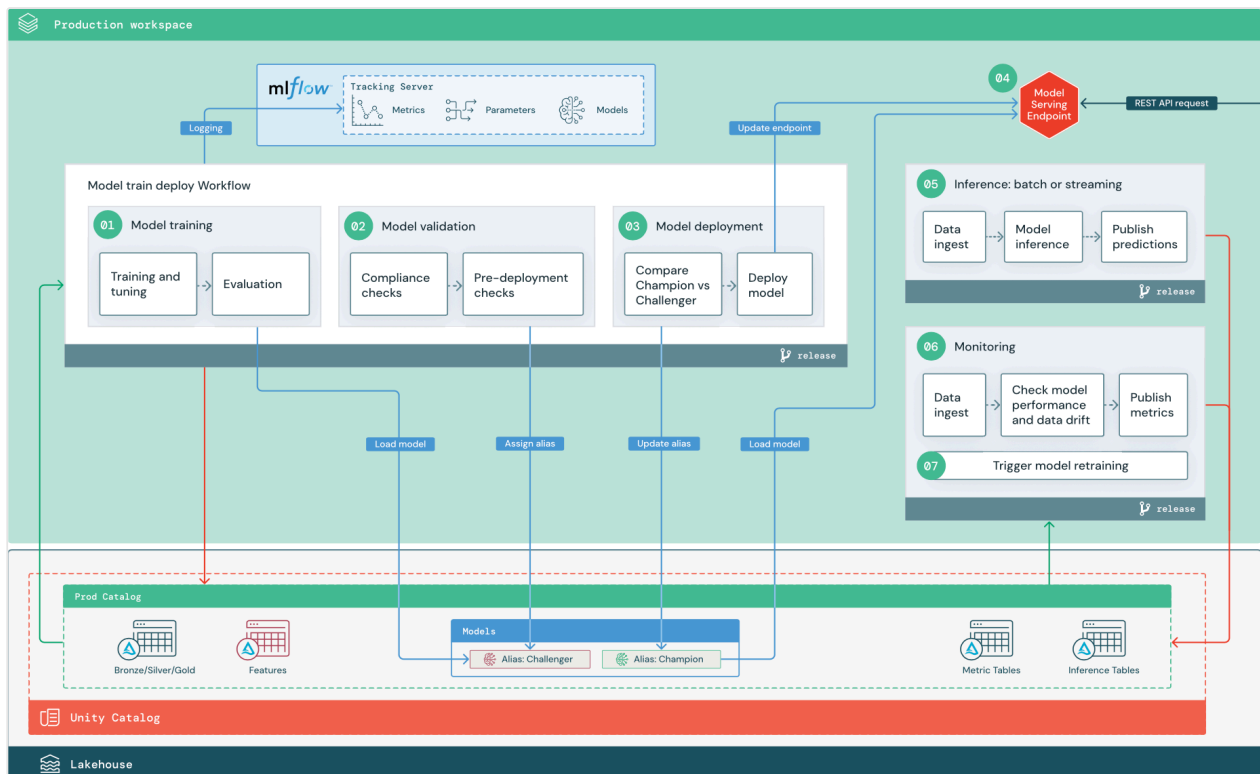
5. Create a release branch

After CI tests have passed and the dev branch is merged into the main branch, the ML engineer creates a release branch, which triggers the CI/CD system to update production jobs.

Production stage

ML engineers own the production environment where ML pipelines are deployed and executed. These pipelines trigger model training, validate and deploy new model versions, publish predictions to downstream tables or applications, and monitor the entire process to avoid performance degradation and instability.

Data scientists typically do not have write or compute access in the production environment. However, it is important that they have visibility to test results, logs, model artifacts, production pipeline status, and monitoring tables. This visibility allows them to identify and diagnose problems in production and to compare the performance of new models to models currently in production. You can grant data scientists read-only access to assets in the production catalog for these purposes.



The numbered steps correspond to the numbers shown in the diagram.

1. Train model

This pipeline can be triggered by code changes or by automated retraining jobs. In this step, tables from the production catalog are used for the following steps.

- **Training and tuning.** During the training process, logs are recorded to the production environment MLflow Tracking server. These logs include model metrics, parameters, tags, and the model itself. If you use feature tables, the model is logged to MLflow using the Databricks Feature Store client, which packages the model with feature lookup information that is used at inference time.

During development, data scientists may test many algorithms and hyperparameters. In the production training code, it's common to consider only the top-performing options. Limiting tuning in this way saves time and can reduce the variance from tuning in automated retraining.

If data scientists have read-only access to the production catalog, they may be able to determine the optimal set of hyperparameters for a model. In this case, the model training pipeline deployed in production can be executed using the

selected set of hyperparameters, typically included in the pipeline as a configuration file.

- **Evaluation.** Model quality is evaluated by testing on held-out production data. The results of these tests are logged to the MLflow tracking server. This step uses the evaluation metrics specified by data scientists in the development stage. These metrics may include custom code.
- **Register model.** When model training is complete, the model artifact is saved as a registered model version at the specified model path in the production catalog in Unity Catalog. The model training task yields a model URI that the model validation task can use. You can use [task values](#) to pass this URI to the model.

2. Validate model

This pipeline uses the model URI from Step 1 and loads the model from Unity Catalog. It then executes a series of validation checks. These checks depend on your organization and use case, and can include things like basic format and metadata validations, performance evaluations on selected data slices, and compliance with organizational requirements such as compliance checks for tags or documentation.

If the model successfully passes all validation checks, you can assign the “Challenger” alias to the model version in Unity Catalog. If the model does not pass all validation checks, the process exits and users can be automatically notified. You can use tags to add key-value attributes depending on the outcome of these validation checks. For example, you could create a tag “model_validation_status” and set the value to “PENDING” as the tests execute, and then update it to “PASSED” or “FAILED” when the pipeline is complete.

Because the model is registered to Unity Catalog, data scientists working in the development environment can load this model version from the production catalog to investigate if the model fails validation. Regardless of the outcome, results are recorded to the registered model in the production catalog using annotations to the model version.

3. Deploy model

Like the validation pipeline, the model deployment pipeline depends on your organization and use case. This section assumes that you have assigned the newly validated model the “Challenger” alias, and that the existing production model has been assigned the “Champion” alias. The first step before deploying the new model is to confirm that it performs at least as well as the current production model.

- **Compare “CHALLENGER” to “CHAMPION” model.** You can perform this comparison offline or online. An offline comparison evaluates both models against a held-out data set and tracks results using the MLflow Tracking server. For real-time model serving, you might want to perform longer running online comparisons, such as A/B tests or a gradual rollout of the new model. If the “Challenger” model version performs better in the comparison, it replaces the current “Champion” alias.

Model Serving and data profiling allow you to automatically collect and monitor inference tables that contain request and response data for an endpoint.

If there is no existing “Champion” model, you might compare the “Challenger” model to a business heuristic or other threshold as a baseline.

The process described here is fully automated. If manual approval steps are required, you can set those up using workflow notifications or CI/CD callbacks from the model deployment pipeline.

- **Deploy model.** Batch or streaming inference pipelines can be set up to use the model with the “Champion” alias. For real-time use cases, you must set up the infrastructure to deploy the model as a REST API endpoint. You can create and manage this endpoint using Model Serving. If an endpoint is already in use for the current model, you can update the endpoint with the new model. Model Serving executes a zero-downtime update by keeping the existing configuration running until the new one is ready.

4. Model Serving

When configuring a Model Serving endpoint, you specify the name of the model in Unity Catalog and the version to serve. If the model version was trained using features from tables in Unity Catalog, the model stores the dependencies for the features and functions. Model Serving automatically uses this dependency graph to look up  Ask Genie

features from appropriate online stores at inference time. This approach can also be used to apply functions for data preprocessing or to compute on-demand features during model scoring.

You can create a single endpoint with multiple models and specify the endpoint traffic split between those models, allowing you to conduct online “Champion” versus “Challenger” comparisons.

5. Inference: batch or streaming

The inference pipeline reads the latest data from the production catalog, executes functions to compute on-demand features, loads the “Champion” model, scores the data, and returns predictions. Batch or streaming inference is generally the most cost-effective option for higher throughput, higher latency use cases. For scenarios where low-latency predictions are required, but predictions can be computed offline, these batch predictions can be published to an online key-value store such as DynamoDB or Cosmos DB.

The registered model in Unity Catalog is referenced by its alias. The inference pipeline is configured to load and apply the “Champion” model version. If the “Champion” version is updated to a new model version, the inference pipeline automatically uses the new version for its next execution. In this way the model deployment step is decoupled from inference pipelines.

Batch jobs typically publish predictions to tables in the production catalog, to flat files, or over a JDBC connection. Streaming jobs typically publish predictions either to Unity Catalog tables or to message queues like Apache Kafka.

6. Data profiling

Data profiling monitors statistical properties, such as data drift and model performance, of input data and model predictions. You can create alerts based on these metrics or publish them in dashboards.

- **Data ingestion.** This pipeline reads in logs from batch, streaming, or online inference.
- **Check accuracy and data drift.** The pipeline computes metrics about the input data, the model's predictions, and the infrastructure performance. Data scientists

specify data and model metrics during development, and ML engineers specify infrastructure metrics. You can also define custom metrics.

- **Publish metrics and set up alerts.** The pipeline writes to tables in the production catalog for analysis and reporting. You should configure these tables to be readable from the development environment so data scientists have access for analysis. You can use Databricks SQL to create monitoring dashboards to track model performance, and set up the monitoring job or the dashboard tool to issue a notification when a metric exceeds a specified threshold.
- **Trigger model retraining.** When monitoring metrics indicate performance issues or changes in the input data, the data scientist may need to develop a new model version. You can set up SQL alerts to notify data scientists when this happens.

7. Retraining

This architecture supports automatic retraining using the same model training pipeline above. Databricks recommends beginning with scheduled, periodic retraining and moving to triggered retraining when needed.

- **Scheduled.** If new data is available on a regular basis, you can create a scheduled job to run the model training code on the latest available data. See [Automating jobs with schedules and triggers](#)
- **Triggered.** If the monitoring pipeline can identify model performance issues and send alerts, it can also trigger retraining. For example, if the distribution of incoming data changes significantly or if the model performance degrades, automatic retraining and redeployment can boost model performance with minimal human intervention. This can be achieved through a SQL alert to check whether a metric is anomalous (for example, check drift or model quality against a threshold). The alert can be configured to use a webhook destination, which can subsequently trigger the training workflow.

If the retraining pipeline or other pipelines exhibit performance issues, the data scientist may need to return to the development environment for additional experimentation to address the issues.

Is this page

as this page helpful?

☐ Yes

☐ No

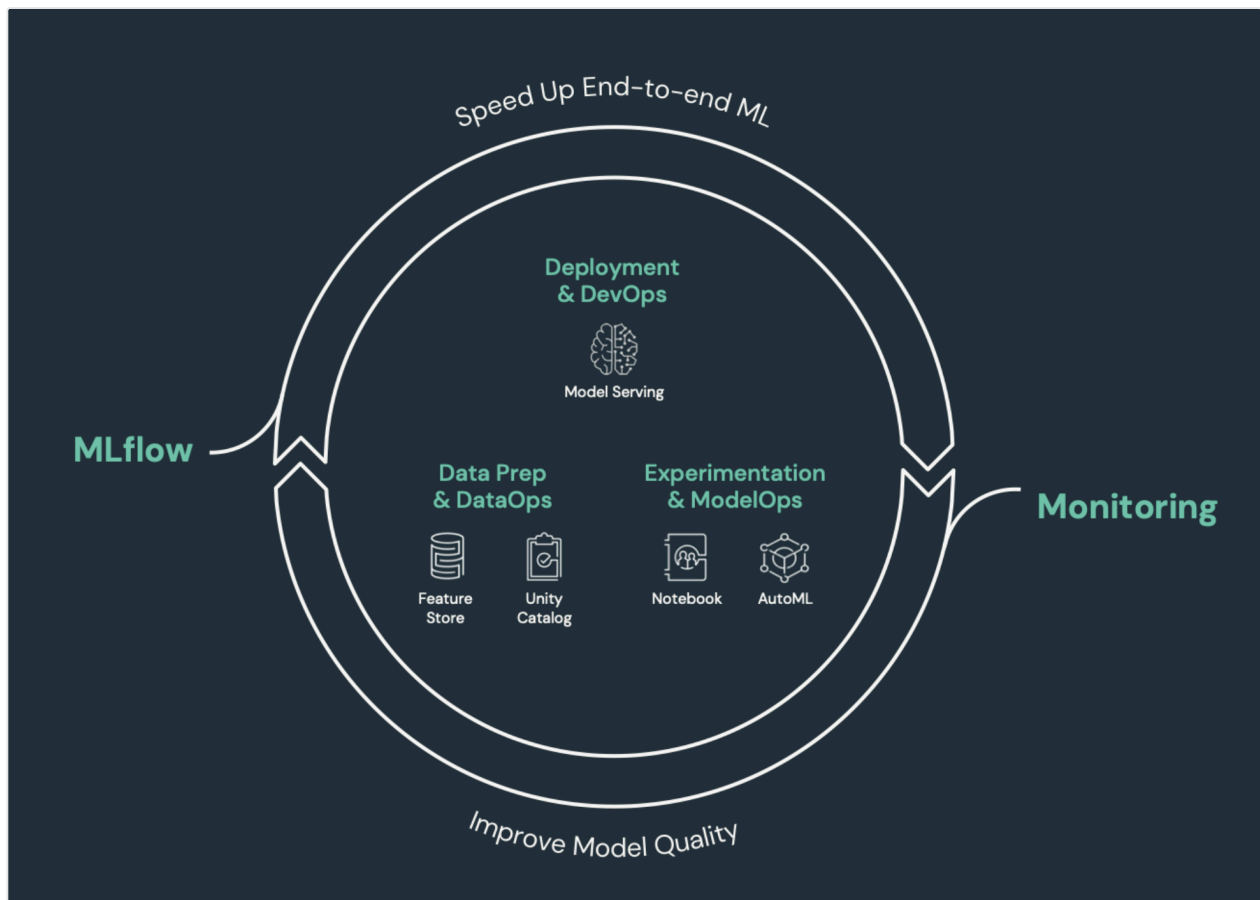
☐ Send feedback

Last updated on **May 11, 2026**

How does Databricks support CI/CD for machine learning?

CI/CD (continuous integration and continuous delivery) refers to an automated process for developing, deploying, monitoring, and maintaining your applications. By automating the building, testing, and deployment of code, development teams can deliver releases more frequently and reliably than manual processes still prevalent across many data engineering and data science teams. CI/CD for machine learning brings together techniques of MLOps, DataOps, ModelOps, and DevOps.

This article describes how Databricks supports CI/CD for machine learning solutions. In machine learning applications, CI/CD is important not only for code assets, but is also applied to data pipelines, including both input data and the results generated by the model.



Machine learning elements that need CI/CD

One of the challenges of ML development is that different teams own different parts of the process. Teams may rely on different tools and have different release schedules. Databricks provides a single, unified data and ML platform with integrated tools to improve teams' efficiency and ensure consistency and repeatability of data and ML pipelines.

In general for machine learning tasks, the following should be tracked in an automated CI/CD workflow:

- Training data, including data quality, schema changes, and distribution changes.
- Input data pipelines.
- Code for training, validating, and serving the model.
- Model predictions and performance.

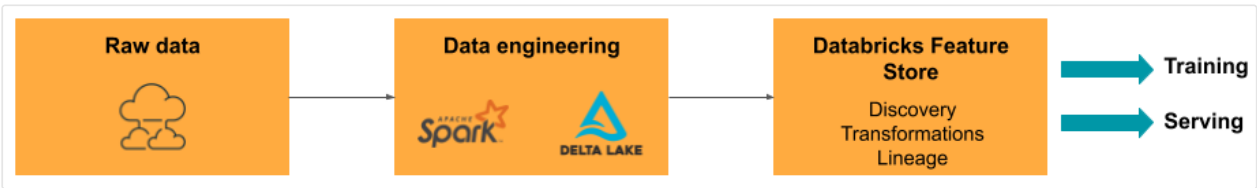
Integrate Databricks into your CI/CD processes

[MLOps](#), DataOps, ModelOps, and DevOps refer to the integration of development processes with “operations” – making the processes and infrastructure predictable and reliable. This set of articles describes how to integrate operations (“ops”) principles into your ML workflows on the Databricks platform.

Databricks incorporates all of the components required for the ML lifecycle including tools to build “configuration as code” to ensure reproducibility and “infrastructure as code” to automate the provisioning of cloud services. It also includes logging and alerting services to help you detect and troubleshoot problems when they occur.

DataOps: Reliable and secure data

Good ML models depend on reliable data pipelines and infrastructure. With the Databricks Data Intelligence Platform, the entire data pipeline from ingesting data to the outputs from the served model is on a single platform and uses the same toolset, which facilitates productivity, reproducibility, sharing, and troubleshooting.



DataOps tasks and tools in Databricks

The table lists common DataOps tasks and tools in Databricks:

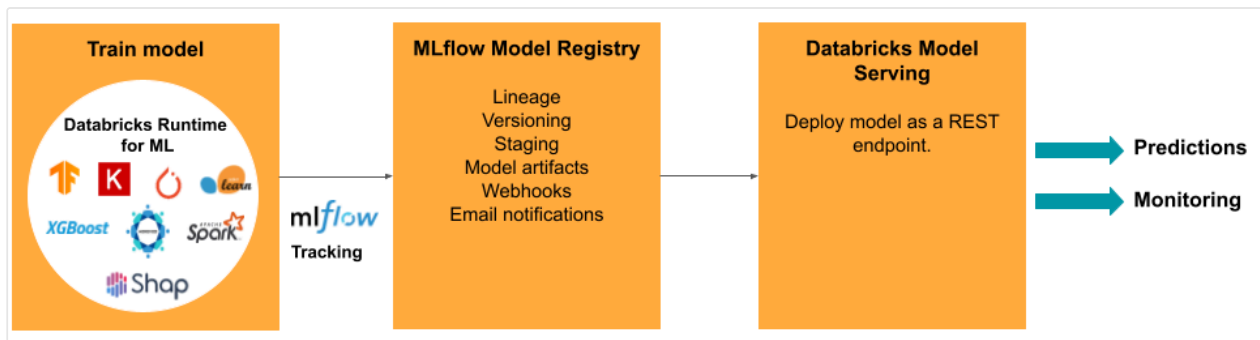
DataOps task	Tool in Databricks
Ingest and transform data	Auto Loader and Apache Spark
Track changes to data including versioning and lineage	Delta tables
	 Ask Genie

DataOps task	Tool in Databricks
Build, manage, and monitor data processing pipelines	Lakeflow Spark Declarative Pipelines
Ensure data security and governance	Unity Catalog
Exploratory data analysis and dashboards	Databricks SQL, Dashboards, and Databricks notebooks
General coding	Databricks SQL and Databricks notebooks
Schedule data pipelines	Lakeflow Jobs
Automate general workflows	Lakeflow Jobs
Create, store, manage, and discover features for model training	Databricks Feature Store
Data monitoring	Data quality monitoring

ModelOps: Model development and lifecycle

Developing a model requires a series of experiments and a way to track and compare the conditions and results of those experiments. The Databricks Data Intelligence Platform includes MLflow for model development tracking and the MLflow Model Registry to manage the model lifecycle including staging, serving, and storing model artifacts.

After a model is released to production, many things can change that might affect its performance. In addition to monitoring the model's prediction performance, you should also monitor input data for changes in quality or statistical characteristics that might require retraining the model.



ModelOps tasks and tools in Databricks

The table lists common ModelOps tasks and tools provided by Databricks:

ModelOps task	Tool in Databricks
Track model development	MLflow model tracking
Manage model lifecycle	Models in Unity Catalog
Model code version control and sharing	Databricks Git folders
No-code model development	AutoML
Model monitoring	Data profiling

DevOps: Production and automation

The Databricks platform supports ML models in production with the following:

- End-to-end data and model lineage: From models in production back to the raw data source, on the same platform.
- Production-level Model Serving: Automatically scales up or down based on your business needs.
- Jobs: Automates jobs and create scheduled machine learning workflows.
- Git folders: Code versioning and sharing from the workspace, also helps teams follow software engineering best practices.

- [Declarative Automation Bundles](#): Automates the creation and deployment of Databricks resources, such as jobs, registered models, and serving endpoints.
- [Databricks Terraform provider](#): Automates deployment infrastructure across clouds for ML inference jobs, serving endpoints, and featurization jobs.

Model serving

For deploying models to production, MLflow significantly simplifies the process, providing single-click deployment as a batch job for large amounts of data or as a REST endpoint on an autoscaling cluster. The integration of Databricks Feature Store with MLflow also ensures consistency of features for training and serving; also, MLflow models can automatically look up features from the Feature Store, even for low latency online serving.

The Databricks platform supports many model deployment options:

- Code and containers.
- Batch serving.
- Low-latency online serving.
- On-device or edge serving.
- Multi-cloud, for example, training the model on one cloud and deploying it with another.

For more information, see [Model Serving](#).

Jobs

[Lakeflow Jobs](#) allow you to automate and schedule any type of workload, from ETL to ML. Databricks also supports integrations with popular third party [orchestrators like Airflow](#).

Git folders

The Databricks platform includes Git support in the workspace to help teams follow software engineering best practices by performing Git operations through the UI. Administrators and DevOps engineers can use APIs to set up automation with their

favorite CI/CD tools. Databricks supports any type of Git deployment including private networks.

For more information about best practices for code development using Databricks Git folders, see [CI/CD workflows with Git integration and Databricks Git folders](#) and [Use CI/CD](#). These techniques, together with the Databricks REST API, let you build automated deployment processes with GitHub Actions, Azure DevOps pipelines, or Jenkins jobs.

Unity Catalog for governance and security

The Databricks platform includes [Unity Catalog](#), which lets admins set up fine-grained access control, security policies, and governance for all data and AI assets across Databricks.

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

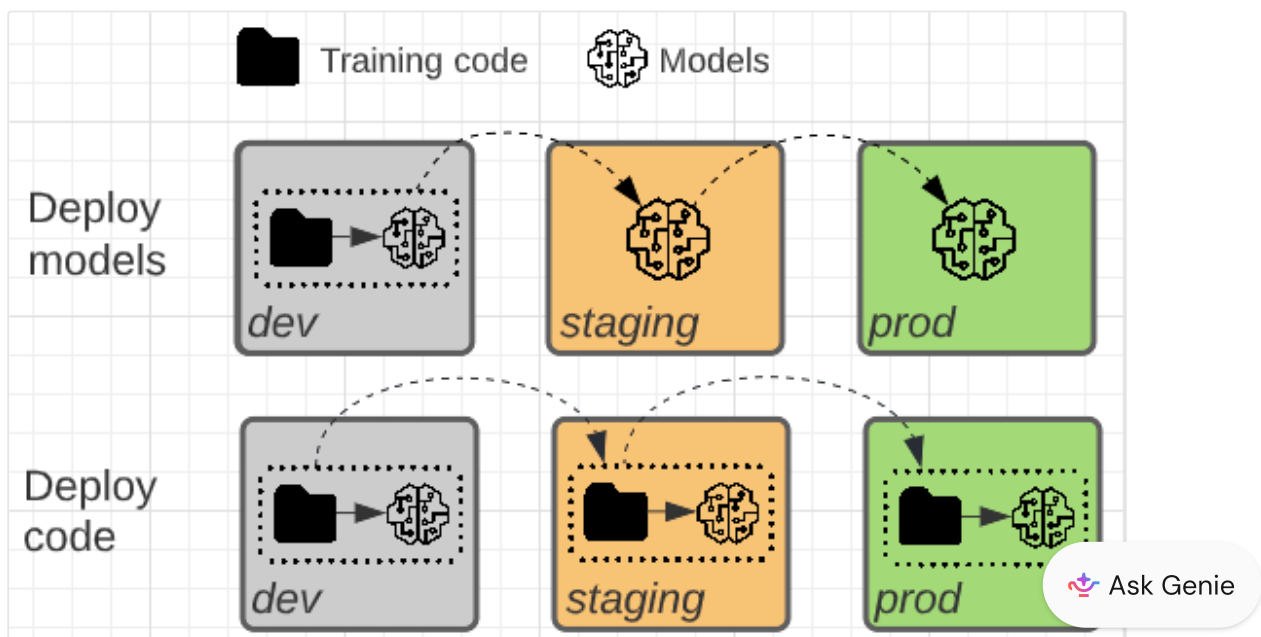
Last updated on **Oct 1, 2024**

Model deployment patterns

This article describes two common patterns for moving ML artifacts through staging and into production. The asynchronous nature of changes to models and code means that there are multiple possible patterns that an ML development process might follow.

Models are created by code, but the resulting model artifacts and the code that created them can operate asynchronously. That is, new model versions and code changes might not happen at the same time. For example, consider the following scenarios:

- To detect fraudulent transactions, you develop an ML pipeline that retrains a model weekly. The code may not change very often, but the model might be retrained every week to incorporate new data.
- You might create a large, deep neural network to classify documents. In this case, training the model is computationally expensive and time-consuming, and retraining the model is likely to happen infrequently. However, the code that deploys, serves, and monitors this model can be updated without retraining the model.



The two patterns differ in whether *the model artifact* or *the training code that produces the model artifact* is promoted towards production.

Deploy code (recommended)

In most situations, Databricks recommends the “deploy code” approach. This approach is incorporated into the [recommended MLOps workflow](#).

In this pattern, the code to train models is developed in the development environment. The same code moves to staging and then production. The model is trained in each environment: initially in the development environment as part of model development, in staging (on a limited subset of data) as part of integration tests, and in the production environment (on the full production data) to produce the final model.

Advantages:

- In organizations where access to production data is restricted, this pattern allows the model to be trained on production data in the production environment.
- Automated model retraining is safer, since the training code is reviewed, tested, and approved for production.
- Supporting code follows the same pattern as model training code. Both go through integration tests in staging.

Disadvantages:

- The learning curve for data scientists to hand off code to collaborators can be steep. Predefined project templates and workflows are helpful.

Also in this pattern, data scientists must be able to review training results from the production environment, as they have the knowledge to identify and fix ML-specific issues.

If your situation requires that the model be trained in staging over the full production dataset, you can use a hybrid approach by deploying code to staging, training the model, and then deploying the model to production. This approach saves training costs in production but adds an extra operation cost in staging.

Deploy models

In this pattern, the model artifact is generated by training code in the development environment. The artifact is then tested in the staging environment before being deployed into production.

Consider this option when one or more of the following apply:

- Model training is very expensive or hard to reproduce.
- All work is done in a single Databricks workspace.
- You are not working with external repos or a CI/CD process.

Advantages:

- A simpler handoff for data scientists
- In cases where model training is expensive, only requires training the model once.

Disadvantages:

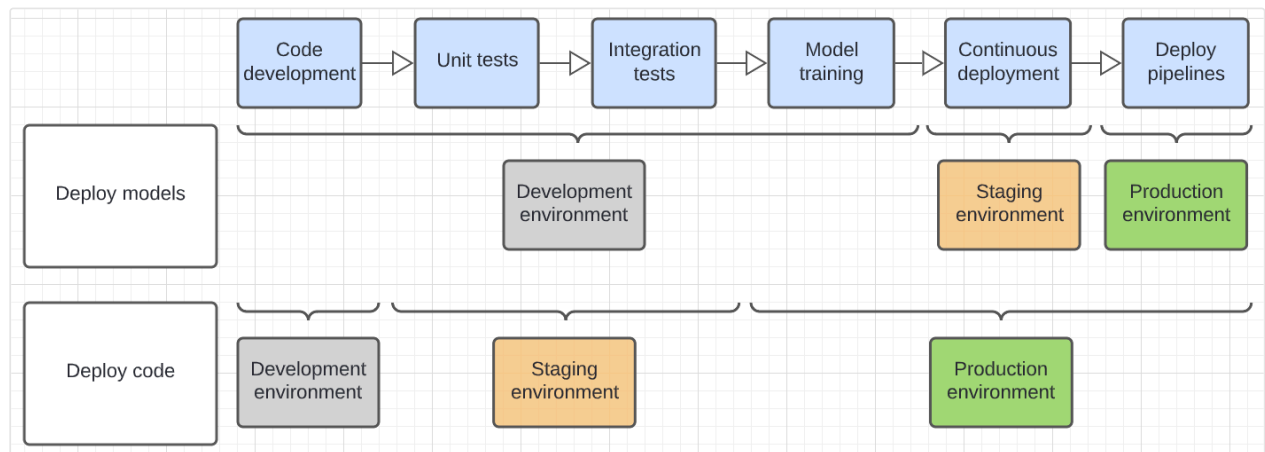
- If production data is not accessible from the development environment (which may be true for security reasons), this architecture may not be viable.
- Automated model retraining is tricky in this pattern. You could automate retraining in the development environment, but the team responsible for deploying the model in production might not accept the resulting model as production-ready.
- Supporting code, such as pipelines used for feature engineering, inference, and monitoring, needs to be deployed to production separately.

Typically an environment (development, staging, or production) corresponds to a catalog in Unity Catalog. For details on how to implement this pattern, see [the upgrade guide](#).

The diagram below contrasts the code lifecycle for the above deployment patterns across the different execution environments.

The environment shown in the diagram is the final environment in which a step is run. For example, in the deploy models pattern, final unit and integration testing is performed in the development environment. In the deploy code pattern, unit tests and

integration tests are run in the development environments, and final unit and integration testing is performed in the staging environment.



Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

Last updated on **May 11, 2026**

MLOps Stacks: model development process as code

This article describes how MLOps Stacks lets you implement the development and deployment process as code in a source-controlled repository. It also describes the benefits of model development on the Databricks Data Intelligence platform, a single platform that unifies every step of the model development and deployment process.

What is MLOps Stacks?

With MLOps Stacks, the entire model development process is implemented, saved, and tracked as code in a source-controlled repository. Automating the process in this way facilitates more repeatable, predictable, and systematic deployments and makes it possible to integrate with your CI/CD process. Representing the model development process as code enables you to deploy the code instead of deploying the model. Deploying the code automates the ability to build the model, making it much easier to retrain the model when necessary.

When you create a project using MLOps Stacks, you define the components of your ML development and deployment process such as notebooks to use for feature engineering, training, testing, and deployment, pipelines for training and testing, workspaces to use for each stage, and CI/CD workflows using GitHub Actions or Azure DevOps for automated testing and deployment of your code.

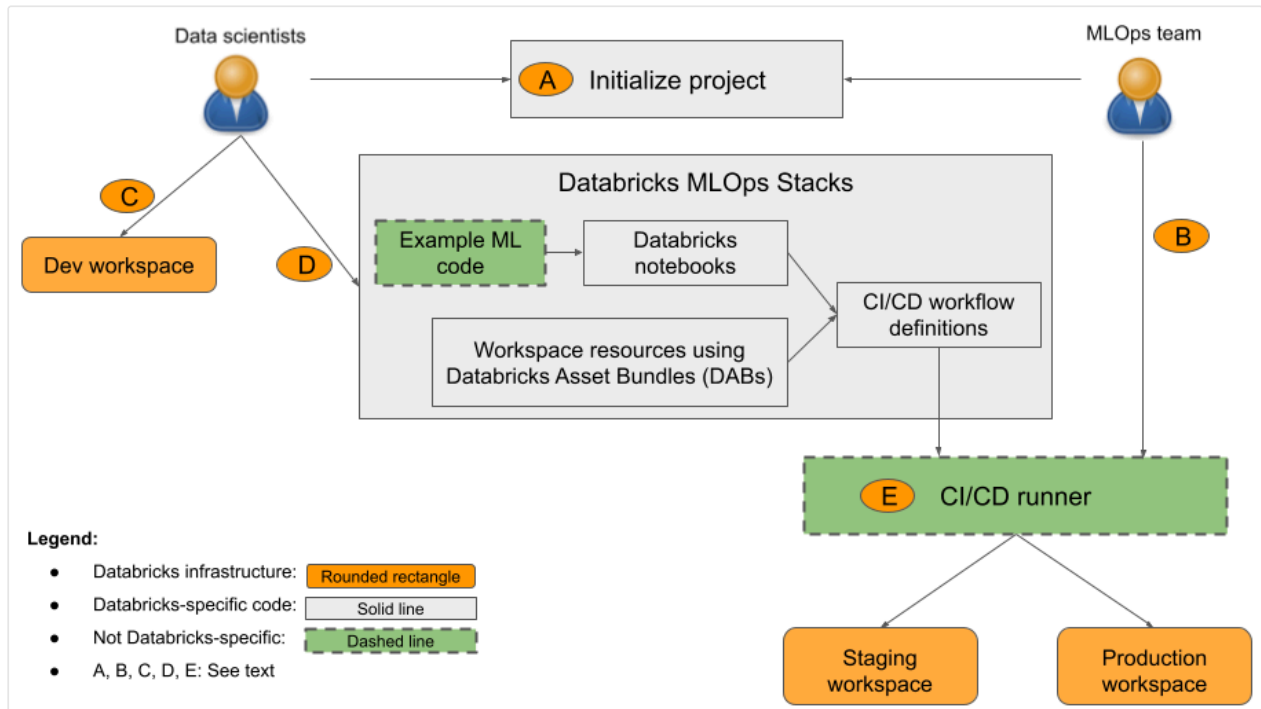
The environment created by MLOps Stacks implements the [MLOps workflow recommended by Databricks](#). You can customize the code to create stacks to match your organization's processes or requirements.

How does MLOps Stacks work?



You use the Databricks CLI to create an MLOps Stack. For step-by-step instructions, see [Declarative Automation Bundles for MLOps Stacks](#).

When you initiate an MLOps Stacks project, the software steps you through entering the configuration details and then creates a directory containing the files that compose your project. This directory, or stack, implements the production MLOps workflow recommended by Databricks. The components shown in the diagram are created for you, and you need only edit the files to add your custom code.



In the diagram:

- A:** A data scientist or ML engineer initializes the project using `databricks bundle init mlops-stacks`. When you initialize the project, you can choose to set up the ML code components (typically used by data scientists), the CI/CD components (typically used by ML engineers), or both.
- B:** ML engineers set up Databricks service principal secrets for CI/CD.
- C:** Data scientists develop models on Databricks or on their local system.
- D:** Data scientists create pull requests to update ML code.
- E:** The CI/CD runner runs notebooks, creates jobs, and performs other tasks in the staging and production workspaces.

Your organization can use the default stack, or customize it as needed to add, remove, or revise components to fit your organization's practices. See the [GitHub repository](#)  Ask Genie

[readme](#) for details.

MLOps Stacks is designed with a modular structure to allow the different ML teams to work independently on a project while following software engineering best practices and maintaining production-grade CI/CD. Production engineers configure ML infrastructure that allows data scientists to develop, test, and deploy ML pipelines and models to production.

As shown in the diagram, the default MLOps Stack includes the following three components:

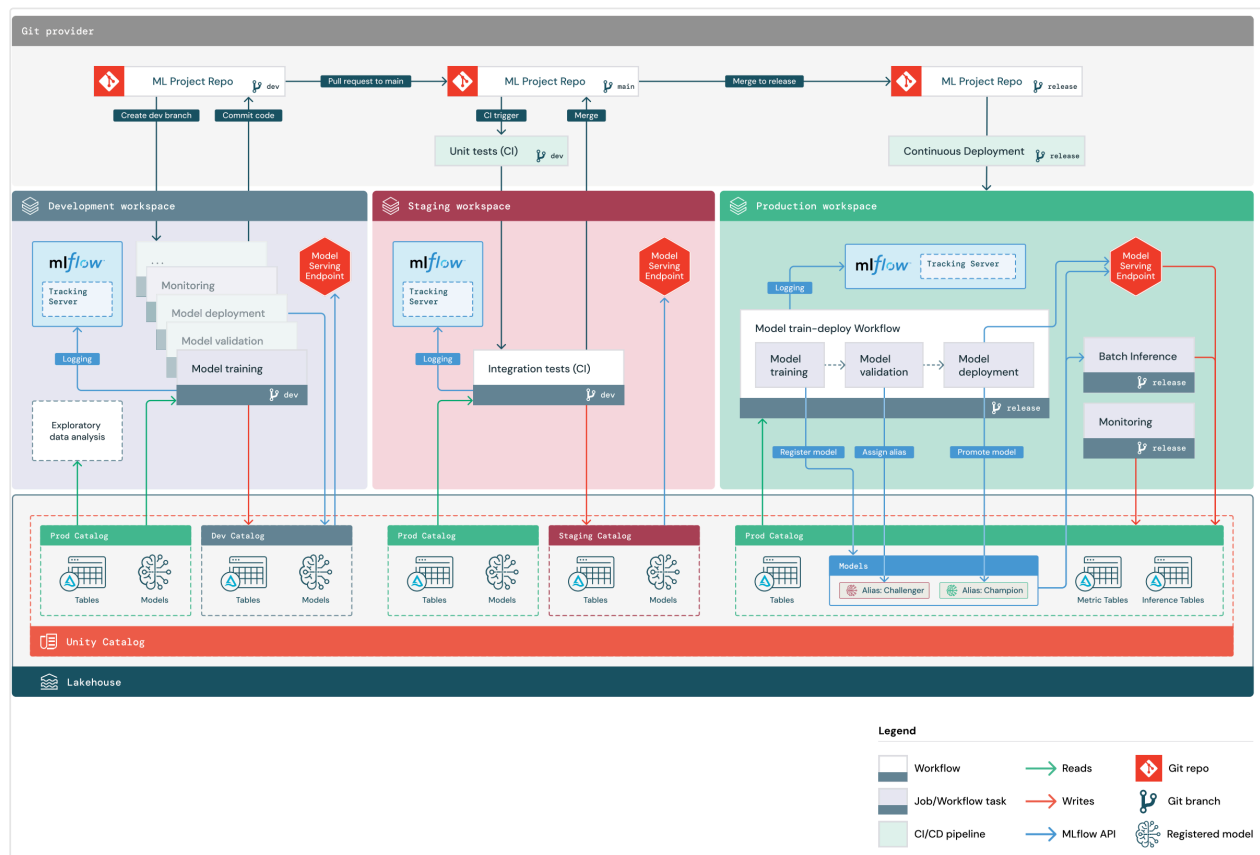
- **ML code.** MLOps Stacks creates a set of templates for an ML project including notebooks for training, batch inference, and so on. The standardized template allows data scientists to get started quickly, unifies project structure across teams, and enforces modularized code ready for testing.
- **ML resources as code.** MLOps Stacks defines resources such as workspaces and pipelines for tasks like training and batch inference. Resources are defined in Declarative Automation Bundles to facilitate testing, optimization, and version control for the ML environment. For example, you can try a larger instance type for automated model retraining, and the change is automatically tracked for future reference.
- **CI/CD.** You can use GitHub Actions or Azure DevOps to test and deploy ML code and resources, ensuring that all production changes are performed through automation and that only tested code is deployed to prod.

MLOps project flow

A default MLOps Stacks project includes an ML pipeline with CI/CD workflows to test and deploy automated model training and batch inference jobs across development, staging, and production Databricks workspaces. MLOps Stacks is configurable, so you can modify the project structure to meet your organization's processes.

The diagram shows the process that is implemented by the default MLOps Stack. In the development workspace, data scientists iterate on ML code and file pull requests (PRs). PRs trigger unit tests and integration tests in an isolated staging Databricks workspace. When a PR is merged to main, model training and batch inference jobs that run in staging immediately update to run the latest code. After you merge a PR into

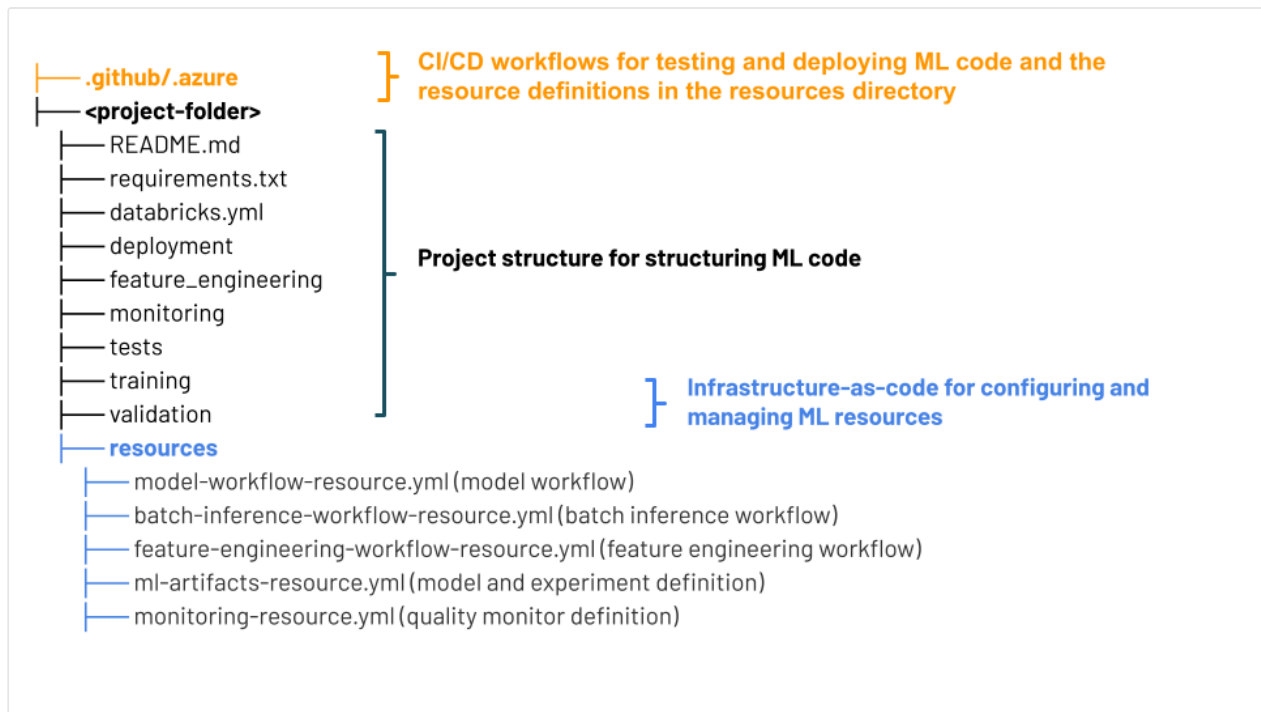
main, you can cut a new release branch as part of your scheduled release process and deploy the code changes to production.



MLOps Stacks project structure

An MLOps Stack uses [Declarative Automation Bundles](#) – a collection of source files that serves as the end-to-end definition of a project. These source files include information about how they are to be tested and deployed. Collecting the files as a bundle makes it easy to co-version changes and use software engineering best practices such as source control, code review, testing, and CI/CD.

The diagram shows the files created for the default MLOps Stack. For details about the files included in the stack, see the documentation on the [GitHub repository](#) or [Declarative Automation Bundles for MLOps Stacks](#).



MLOps Stacks components

A “stack” refers to the set of tools used in a development process. The default MLOps Stack takes advantage of the unified Databricks platform and uses the following tools:

Component	Tool in Databricks
ML model development code	Databricks notebooks, MLflow
Feature development and management	Feature engineering
ML model repository	Models in Unity Catalog
ML model serving	Model Serving
Infrastructure-as-code	Declarative Automation Bundles
Orchestrator	Lakeflow Jobs
CI/CD	GitHub Actions, Azure DevOps
 Ask Genie	

Component	Tool in Databricks
Data and model performance monitoring	Data profiling

Next steps

To get started, see [Declarative Automation Bundles for MLOps Stacks](#) or the [Databricks MLOps Stacks repository on GitHub](#).

Is this page

as this page helpful?

☐ Yes

☐ No

☐ Send feedback

Last updated on **May 11, 2026**

LLMOps workflows on Databricks

This article complements [MLOps workflows on Databricks](#) by adding information specific to LLMOps workflows. For more details, see [The Big Book of MLOps](#).

How does the MLOps workflow change for LLMs?

LLMs are a class of natural language processing (NLP) models that have significantly surpassed their predecessors in size and performance across a variety of tasks, such as open-ended question answering, summarization, and execution of instructions.

Development and evaluation of LLMs differs in some important ways from traditional ML models. This section briefly summarizes some of the key properties of LLMs and the implications for MLOps.

Key properties of LLMs	Implications for MLOps
LLMs are available in many forms. • General proprietary and OSS models that are accessed using paid APIs. • Off-the-shelf open source models that vary from general to specific applications.	Development process: Projects often develop incrementally, starting from existing, third-party or open source models and ending with custom fine-tuned models.  Ask Genie

Key properties of LLMs	Implications for MLOps
• Custom models that have been fine-tuned for specific applications. • Custom pre-trained applications.	
Many LLMs take general natural language queries and instructions as input. Those queries can contain carefully engineered prompts to elicit the desired responses.	Development process: Designing text templates for querying LLMs is often an important part of developing new LLM pipelines. Packaging ML artifacts: Many LLM pipelines use existing LLMs or LLM serving endpoints. The ML logic developed for those pipelines might focus on prompt templates, agents, or chains instead of the model itself. The ML artifacts packaged and promoted to production might be these pipelines, rather than models.
Many LLMs can be given prompts with examples, context, or other information to help answer the query.	Serving infrastructure: When augmenting LLM queries with context, you might use additional tools such as vector indexes to search for relevant context.
Third-party APIs provide proprietary and open-source models.	API governance: Using centralized API governance provides the ability to easily switch between API providers.
 Ask Genie	

Key properties of LLMs	Implications for MLOps
LLMs are very large deep learning models, often ranging from gigabytes to hundreds of gigabytes.	Serving infrastructure: LLMs might require GPUs for real-time model serving, and fast storage for models that need to be loaded dynamically. Cost/performance tradeoffs: Because larger models require more computation and are more expensive to serve, techniques for reducing model size and computation might be required.
LLMs are hard to evaluate using traditional ML metrics since there is often no single “right” answer.	Human feedback: Human feedback is essential for evaluating and testing LLMs. You should incorporate user feedback directly into the MLOps process, including for testing, monitoring, and future fine-tuning.

Commonalities between MLOps and LLMOps

Many aspects of MLOps processes do not change for LLMs. For example, the following guidelines also apply to LLMs:

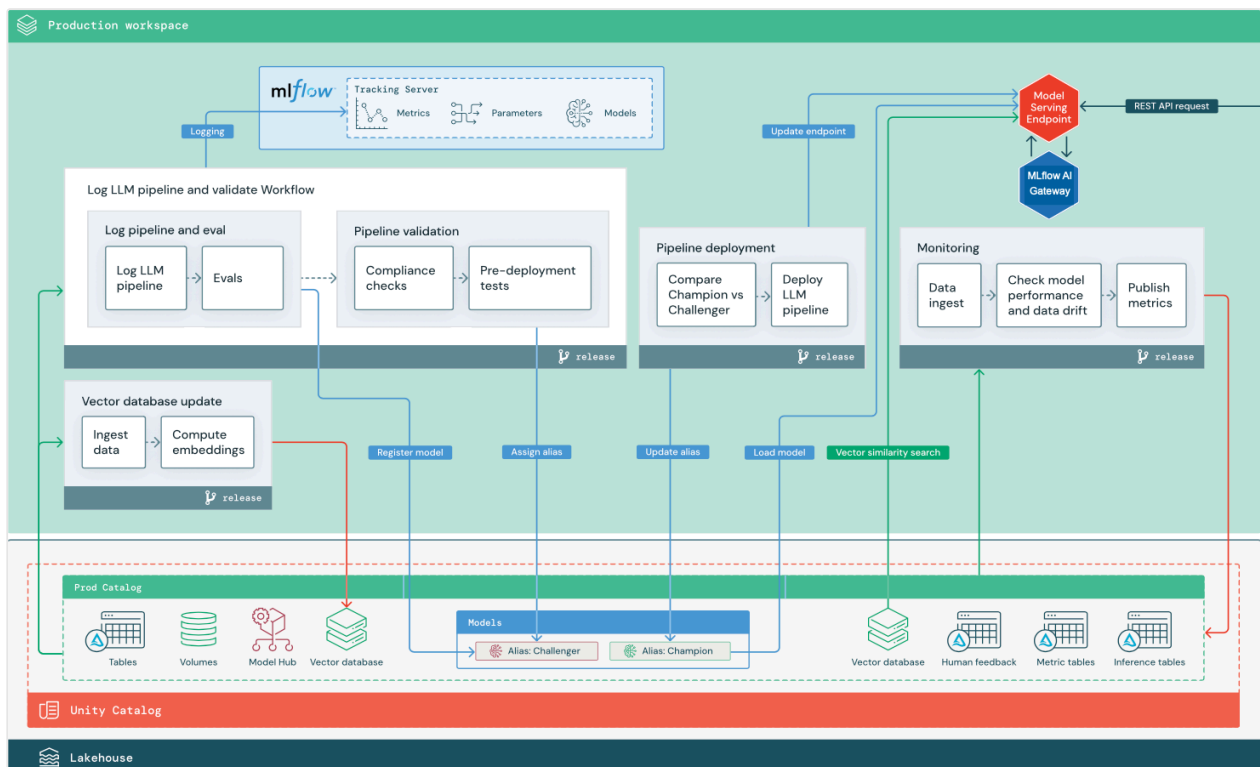
- Use separate environments for development, staging, and production.
- Use Git for version control.
- Manage model development with MLflow, and use [Models in Unity Catalog](#) to manage the model lifecycle.
- Store data in a lakehouse architecture using Delta tables.
- Your existing CI/CD infrastructure should not require any changes.
- The modular structure of MLOps remains the same, with pipelines for featurization, model training, model inference, and so on.

Reference architecture diagrams

This section uses two LLM-based applications to illustrate some of the adjustments to the [reference architecture of traditional MLOps](#). The diagrams show the production architecture for 1) a retrieval-augmented generation (RAG) application using a third-party API, and 2) a RAG application using a self-hosted fine-tuned model. Both diagrams show an optional vector database — this item can be replaced by directly querying the LLM through the Model Serving endpoint.

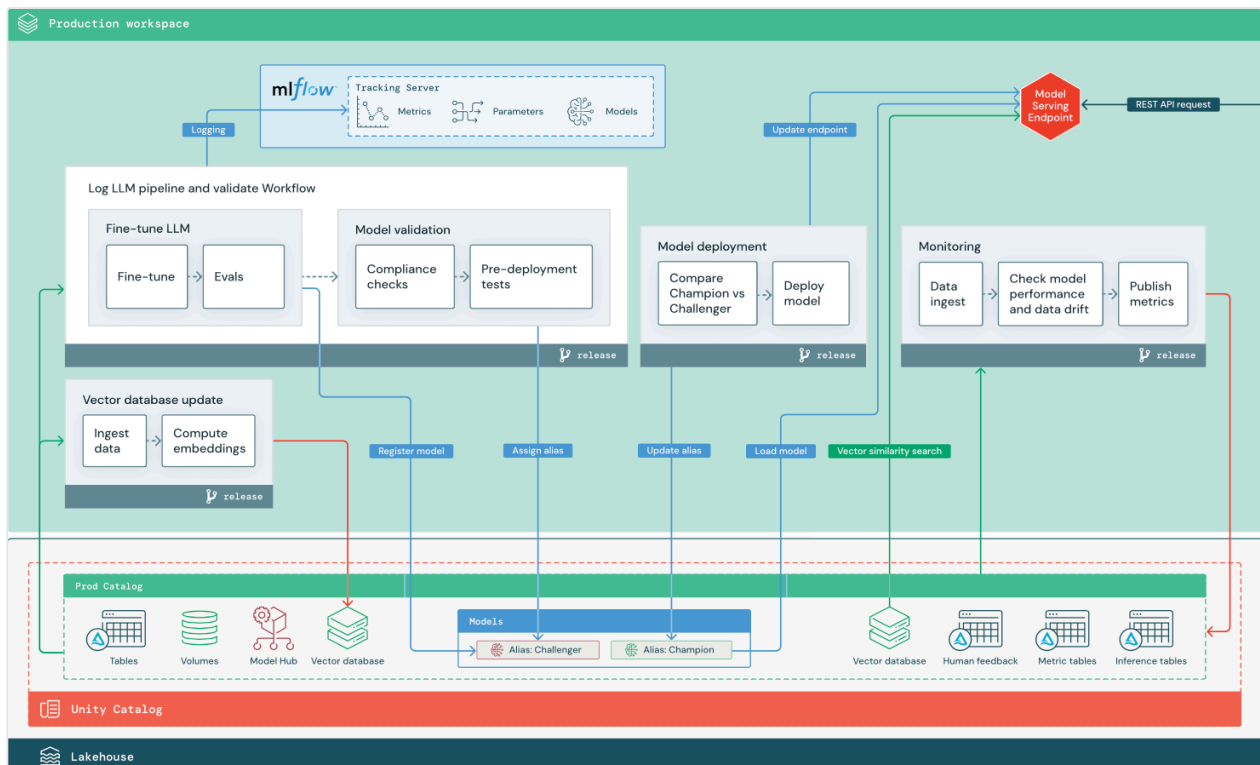
RAG with a third-party LLM API

The diagram shows a production architecture for a RAG application that connects to a third-party LLM API using Databricks External Models.



RAG with a fine-tuned open source model

The diagram shows a production architecture for a RAG application that fine-tunes an open source model.



LLMOps changes to MLOps production architecture

This section highlights the major changes to the MLOps reference architecture for LLMOps applications.

Model hub

LLM applications often use existing, pretrained models selected from an internal or external model hub. The model can be used as-is or fine-tuned.

Databricks includes a selection of high-quality, pre-trained foundation models in Unity Catalog and in Databricks Marketplace. You can use these pre-trained models to access state-of-the-art AI capabilities, saving you the time and expense of building your own custom models. For details, see [Access generative AI and LLM models from Unity Catalog](#).

Vector index

Some LLM applications use vector indexes for fast similarity searches, for example to provide context or domain knowledge in LLM queries. Databricks provides an integrated vector search functionality that lets you use any Delta table in Unity Catalog as a vector index. The vector search index automatically syncs with the Delta table. For details, see [Vector Search](#).

You can create a model artifact that encapsulates the logic to retrieve information from a vector index and provides the returned data as context to the LLM. You can then log the model using the MLflow LangChain or PyFunc model flavor.

Fine-tune LLM

Because LLM models are expensive and time-consuming to create from scratch, LLM applications often fine-tune an existing model to improve its performance in a particular scenario. In the reference architecture, fine-tuning and model deployment are represented as distinct Lakeflow Jobs. Validating a fine-tuned model before deploying is often a manual process.

Databricks provides Foundation Model Fine-tuning, which lets you use your own data to customize an existing LLM to optimize its performance for your specific application. For details, see [Foundation Model Fine-tuning](#).

Model serving

In the RAG using a third-party API scenario, an important architectural change is that the LLM pipeline makes external API calls, from the Model Serving endpoint to internal or third-party LLM APIs. This adds complexity, potential latency, and additional credential management.

Databricks provides Model Serving, which provides a unified interface to deploy, govern, and query AI models. For details, see [Model Serving](#).

Human feedback in monitoring and evaluation

Human feedback loops are essential in most LLM applications. Human feedback should be managed like other data, ideally incorporated into monitoring based on near real-time streaming.

The MLflow review app helps you gather feedback from human reviewers. For details, see [Human feedback](#).

Is this page

as this page helpful?

 Yes

 No

 Send feedback